

# ECMM409 INDIVIDUAL REPORT FOR THE GROUP PROJECT

Student Number: 740090003

## 1.0 CONTRIBUTION TO THE PROJECT

### 1.1 Context

In our team, we decided that each member would research and implement nature-inspired algorithms to solve the Traveling Thief Problem (TTF). After comparing our results, the algorithm that performed the best would be selected as the final approach.

My contribution to the project involved researching, introducing, designing, and implementing the Non-dominated Sorting Genetic Algorithm II (NSGA-II) combined with an Ant Colony Optimization (ACO) TSP solver. NSGA-II was chosen for its ability to efficiently handle multi-objective optimization problems, particularly in complex scenarios like TTF. Its integration with ACO leveraged the latter's strength in generating high-quality travelling salesman problem (TSP) solutions, providing a synergistic approach that outperformed other tested methods in the team in terms of both solution quality and computational efficiency. This algorithm was selected by the team as the final approach due to its superior performance. I executed the algorithm to solve all nine instances of the TTF presented in the competition, ensuring the results adhered to the competition rules and formatting requirements. I shared the result files including the values of the objective functions (profit and time), the picking plan and the tour for all constructed solutions in each problem instance using the appropriate file extension as stated in the competition with my team for submission and performed various experiments to optimize the algorithm's parameters. Additionally, I introduced data handling and algorithm optimization techniques to tackle computational challenges, particularly for large datasets like fnl4461 and pla33810 instances. My approach combined NSGA-II, ACO TSP solvers, and a profit-to-weight ratio sorting 0-1 Knapsack solver.

### 1.2 Algorithm Approach

#### 1.2.1 Graph Construction

The construction graph represented the cities(nodes) and their interconnections(edges). The thief's tour begins at node 1 (the starting point), visiting all cities exactly once before

returning to node 1. Items were available for selection at every city except the starting node.

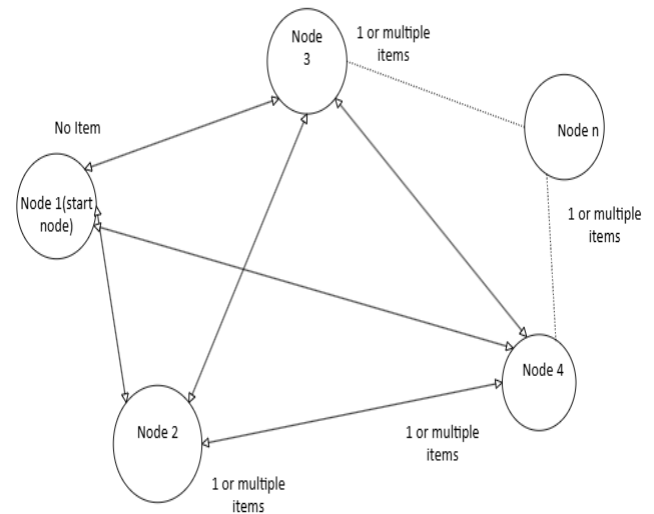


Figure 1. Construction graph

#### 1.2.2 Algorithm Description

##### NSGA-II

NSGA-II is a multi-objective evolutionary algorithm (MOEA) developed by Deb et al. (2002). It uses non-dominated sorting and crowding distance techniques to rank and select the best solutions in each generation. This algorithm was adapted by Blank et al. (2017) for solving the TTF. The NSGA-II algorithm that I implemented uses an ACO TSP solver and a profit-to-weight ratio Knapsack solver to independently solve the TSP and KP components of the TTF. However, the calculation for velocity still ensures that the two components are interdependent.

The NSGA-II algorithm creates an initial population using the solvers to generate routes and picking plans for each parent. The quality of this initial population significantly impacts the algorithm's performance by influencing the convergence speed. In addition, the algorithm benefits from NSGA-II natural tendency to explore diverse sets of

solutions, thereby balancing exploration and exploitation in subsequent generations. Through crossover and mutation, children solutions are generated. The elitist algorithm evaluates parents and children using non-dominated sorting and crowding distance, selecting the best solutions for the next generation. This iterative process continues until a specified number of generations is reached.

### 1.2.3 Route Construction with ACO TSP Solver

The ant starts its journey at node 1 and selects the next city from the unvisited cities based on a probability determined by the pheromone level and a distance heuristic. This process continues until all cities are visited, completing the tour. The pheromone levels are updated after each tour. The distance heuristic prioritizes cities closer to their current position as the objective is to minimize the total route distance.

### 1.2.4 Profit-to-Weight Ratio Knapsack Solver

The Knapsack solver selects items based on their profit-to-weight ratio until the capacity constraint is reached. A random capacity, between 0 and the problem's constraint, is generated for each parent solution. Extreme capacity values were introduced with specific probabilities to promote solution diversity.

The combination of ACO and the Knapsack solver provides high-quality initial solutions for the NSGA-II algorithm. This approach balances the exploration of the solution space and the exploitation of promising regions, enabling efficient identification of near-optimal solutions.

#### Representations and Objective Functions

- Route: A permutation vector representing all cities. The route always starts with node 1.
- Picking Plan: A binary array where each bit indicates whether an item is selected.
- Objective Functions:
  - Profit: Total profit of selected items.
  - Time: Time taken to complete the tour, considering the weight of selected items.

#### Formula for Travel Time

Time = distance / velocity( $V(q)$ ), where:

- $V(q) = V_{\max} - (q/Q)(V_{\max} - V_{\min})$
- $q$  = total weight of selected items.

- $Q$  = capacity constraint.
- $V_{\max} = 1, V_{\min} = 0$  (project parameters).

The goal is to find solutions that maximize profit and minimize time.

## 2.0 ALGORITHM SURVEY

Team members explored various nature-inspired optimization approaches for the TTF, including:

- Memetic Algorithm (MA2B)
- Ant Colony Optimization (ACO)
- Particle Swarm Optimization (PSO)
- NSGA-II with ACO and Knapsack Solver
- Memetic Algorithm with Tabu Search (MATLS)
- Genetic Algorithm

These methods were chosen based on prior research applications to multi-objective problems, including traveling thief problem (TTF). For example, Blank et al. (2017) utilized NSGA-II to solve the TTF problem. Yafrani and Ahiod (2018) also applied the MA2B algorithm to solve the TTF problem. Mei (2015) used the MATLS (Memetic Algorithm with Two-Staged Local Search) approach to address the traveling thief problem. Zouari et al. (2019) applied three variations of the Ant Colony System to solve the problem, while Wagner (2018) used swarm intelligence techniques. The standalone Ant Colony Optimization and Particle Swarm Optimization methods implemented by my team members focused on a single objective function, Profit-Time, for problem resolution. The other approaches used bi-objective functions similar to those found in the NSGA-II algorithm separately evaluating time and profit objective functions. All algorithms employed binary array representation for the picking plan and a permutation vector for the tour route, as detailed in the competition and NSGA-II approach. After implementation and comparison, the NSGA-II combined with ACO and Knapsack solver outperformed the others in terms of performance and computational efficiency.

**Table 1 shows normalized hypervolume values for the a280\_n2790 instance:**

| Algorithms | Normalised Hypervolume values |
|------------|-------------------------------|
| NSGA-II    | 0.9900                        |
| PSO        | 0.9795                        |
| ACO        | 0.9167                        |
| GA         | 0.8650                        |
| MA2B       | 0.6155                        |
| MATL       | 0.5283                        |

### 3.0 EXPERIMENTATION PROCESS

I conducted experiments with NSGA-II, adjusting parameters to balance solution quality and computational time. Initial experiments with standalone NSGA-II produced suboptimal results compared to competition benchmarks. I also performed experiments with an independent ACO TSP solver to optimize its parameters. Final parameters selected included:

- Pheromone Initialization: 1
- Evaporation Rate: 0.3
- Beta: 5
- Pheromone deposit: 100

For the NSGA-II algorithm, I used:

- Order Crossover (OX) for routes.
- Bitflip Mutation for picking plans.
- Single-Point Crossover to generate diverse picking plans.

Data handling optimizations that I introduced and implemented for the team included the use of multi-threading to parallelize data processing, reducing runtime significantly. To manage memory usage effectively, I introduced and implemented techniques such as leveraging NumPy's memory-mapped arrays (`np.memmap`) and creation of a distance.dat distance matrix file stored in the computer system storage disk, enabling the processing of large datasets without exhausting system RAM. The `np.memmap` function ensures that data are only provided to the algorithm temporarily as at when needed. This approach freed over 8.5 gigabytes memory space that would have been

taking up by the distance matrix in the system's RAM. This allowed me to successfully implement the algorithm to solve the problem for the pla33810 problem instances. Prior to the implementation of my solutions, I and other team members were getting memory errors when we tried to run the algorithm on our computer systems. Furthermore, I introduced and implemented the use of a float16 precision array to store the pheromone matrix for the pla33810 problem instances, reducing the memory size of the 33,810 x 33,810 matrix from over 8.5 gigabytes to approximately 2.13 gigabytes. Additionally, I also introduced and implemented approach to minimize computational time by employing techniques that reduced redundant computations, such as filling only the upper diagonal of the distance matrix and leveraging route symmetry to populate the lower diagonal, particularly for pla33810 problem instances. For each problem instance category (a280, fnl4461, pla33180), I also changed the parameters to adjust for computational time. The algorithm parameter values and computational time for each problem instance category can be seen in table 2

**Table 2 showing algorithm parameters and computational details for different problem instance categories**

| Problem Instance | Population size | Number of generations | Number of Fitness Evaluations | Computational Time   |
|------------------|-----------------|-----------------------|-------------------------------|----------------------|
| a280             | 200             | 1000                  | 200,000                       | 35-45 minutes        |
| fnl4461          | 50              | 1000                  | 50,000                        | 1hr-2hrs             |
| pla33810         | 26              | 100                   | 2600                          | 11hrs - 14hrs 25 min |

While the computational time is long for pla33810 problem instances due to the sheer size of the data, this is relatively faster than the average known time for other algorithms that have been used for the pla33810 instances in previous research. Wagner et al. (2018) mentioned that calculations for the pla33810 instances even exceeded 20 hours in their research. In another experiment that I performed using the algorithm to solve problem instance pla33810-n338090, setting the number of generations in the algorithm to 1000, it took a total time of around 17hours 10 minutes for the algorithm to run. However, I was able to get higher quality solutions with all solutions in the final generation being non-dominated solutions.

I explored several other approaches that ultimately proved unsuitable for this project. For instance, I attempted to use

the Lin-Kernighan algorithm for solving the TSP component, but it ran for extended periods without producing results, even for the relatively small a280 problem instances. Similarly, I experimented with dynamic programming for the 0-1 Knapsack component, but it required excessive computation time to generate solutions. A similar issue arose when I tried using a two-opt mutation operator for the TSP, as it also struggled with efficiency. I also tried using the one-opt mutation operator for the TSP, but it did not improve the quality of solutions that the algorithm generated.

## 4.0 TEAMWORKING PROCESS

One of the challenges we faced was ensuring that everyone adopted a consistent approach while working with different algorithms because we all had our own ideas on how to approach the problem. For instance, some team members wanted to use the rent ratio while others preferred the bi-objective approach. Another challenge was scheduling meetings at times that allowed all team members to participate.

The most enjoyable part of the research was engaging in critical discussions and exchanging ideas with team members to determine the most effective approach to solving the problem. For example, we debated the appropriate stopping criteria to use for a fair comparison of results and decided on the number of fitness evaluations to adopt for different problem instances, taking computational time into account. I had a detailed discussion with Butt Talha regarding the use of bitflip as a mutation operator in my algorithm, as he was also working on another version of the genetic algorithm (MA2B).

Each team member contributed by researching and implementing their respective algorithms, which enabled us to explore a diverse range of nature-inspired techniques before selecting the final approach. Nicholas Hartono worked on the Memetic Algorithm with Tabu Search, Mosalpuri Ravi focused on Particle Swarm Optimization, Patel Pallav implemented Ant Colony Optimization, and Chakravarty Rishik applied Genetic Algorithm to solve the optimization problem.

In addition to researching and implementing the algorithms, the team divided various tasks such as report writing, setting up the GitHub repository to share project documents, taking meeting minutes, plotting graphs, collating results to align with the competition's format, and scheduling meetings. All team members were actively involved in the meetings and contributed to these shared responsibilities.

## 5.0 FUTURE RESEARCH

I plan to explore using ACO for the Knapsack problem to enhance profit maximization with advanced computational resources. ACO could address the current limitations of heuristic-based Knapsack solvers by providing a more dynamic, iterative solution process. This approach has the potential to improve solution diversity and accuracy by leveraging pheromone trails and heuristic information to guide item selection. However, it will be important to mitigate its long computational requirements by using more powerful computational systems, integrating parallel processing techniques or hybridizing it with other algorithms to balance efficiency and performance. This approach was not adopted in this project due to computational constraints.

## 6.0 CONCLUSION

In this group project, my primary contribution involved the design and implementation of NSGA-II integrated with ACO, which proved to be the most effective solution based on both solution quality and computational efficiency among all algorithms implemented in the team. Through various experiments, parameter tuning, and algorithm optimizations, the NSGA-II combined with Ant Colony optimization TSP solver and a Profit to weight ratio heuristic Knapsack solver outperformed others in terms of convergence speed, solution diversity, and overall performance.

The algorithm achieved strong results across different problem instances, achieving high normalized hypervolume values, indicating its robustness in solving multi-objective optimization problems. The team also demonstrated effective collaboration, sharing tasks such as algorithm development, experimentation, and report writing, while coordinating efficiently through meetings and shared responsibilities. Moving forward, integrating parallel processing and using Ant Colony optimization algorithm for 0-1 Knapsack solver could also enhance the performance of the algorithm implemented in this project without affecting computational efficiency, paving the way for future improvements in solving similar complex optimization challenges.

This project provided valuable insights into the strengths and limitations of different optimization techniques and highlighted the effectiveness of NSGA-II combined with ACO in solving small and large-scale case instances of multi-objective problems like TTF.

## 7.0 REFERENCES

El Mohamed El Yafrani, Belaïd Ahiod, Population-based vs. Single-solution Heuristics for the Travelling Thief Problem, 2016, 9781450342063, Association for Computing Machinery, New York, NY, USA, [hTTFs://doi.org/10.1145/2908812.2908847](https://doi.org/10.1145/2908812.2908847).

Julian Blank, Solving the Bi-objective Traveling Thief Problem with Multi-objective Evolutionary Algorithms,

Kalyanmoy Deb, Associate Member, IEEE, Amrit Pratap, Sameer Agarwal, and T. Meyarivan, IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 6, NO. 2, APRIL 2002

Kalyanmoy Deb, and Sanaz Mostaghim, Otto-von-Guericke University, 39106 Magdeburg, Germany [julian.blank@gmx.de](mailto:julian.blank@gmx.de), [sanaz.mostaghim@ovgu.de](mailto:sanaz.mostaghim@ovgu.de) 2 Michigan State University, East Lansing, MI 48824, USA [kdeb@egr.msu.edu](mailto:kdeb@egr.msu.edu) [hTTF://www.research-blank.de](http://www.research-blank.de) [hTTF://www.is.ovgu.de/Team/Sanaz+Mostaghim.html](http://www.is.ovgu.de/Team/Sanaz+Mostaghim.html) [hTTF://www.egr.msu.edu/~kdeb/](http://www.egr.msu.edu/~kdeb/)

Mei Y., Li X., Salim F., Yao X., Heuristic Evolution with Genetic Programming for Traveling Thief Problem, May 2015, DOI: ,10.1109/CEC.2015.7257230, Conference: 2015 IEEE Congress on Evolutionary Computation (CEC)At: Sendai, Japan.

Wagner, M., Lindauer, M., Mısıır, M. *et al.* A case study of algorithm selection for the traveling thief problem. *Journal of Heuristics* **24**, 295–320 (2018). [hTTFs://doi.org/10.1007/s10732-017-9328-y](https://doi.org/10.1007/s10732-017-9328-y)

Wagner, M. Stealing Items More Efficiently with Ants: A Swarm Intelligence Approach to the Travelling Thief Problem, *Lecture Notes in Computer Science Swarm Intelligence*, 2016, p. 273-281, Springer International Publishing, [hTTFs://doi.org/10.1007/978-3-319-44427-7\\_25](https://doi.org/10.1007/978-3-319-44427-7_25)

Wiem Zouari, Ines Alaya, and Moncef Tagina. 2019. A new hybrid ant colony algorithms for the traveling thief problem. In Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '19). Association for Computing Machinery, New York, NY, USA, 95–96. [hTTFs://doi.org/10.1145/3319619.3326785](https://doi.org/10.1145/3319619.3326785)

## Template for student declarations in AI-supported and AI-integrated assessments

*When submitting your assessment, you must include the following declaration, ticking all that apply:*

*AI-supported/AI-integrated use is permitted in this assessment. I acknowledge the following uses of GenAI tools in this assessment:*

- ✓ ☐ *I have used GenAI tools for developing ideas.*
- ✓ ☐ *I have used GenAI tools to assist with research or gathering information.*
- ✓ ☐ *I have used GenAI tools to help me understand key theories and concepts.*
- ☐ *I have used GenAI tools to identify trends and themes as part of my data analysis.*
- ☐ *I have used GenAI tools to suggest a plan or structure for my assessment.*
- ✓ ☐ *I have used GenAI tools to give me feedback on a draft.*
- ☐ *I have used GenAI tool to generate image, figures or diagrams.*
- ✓ ☐ *I have used GenAI tools to proofread and correct grammar or spelling errors.*
- ☐ *I have used GenAI tools to generate citations or references.*
- ☐ *Other [please specify]*

*I declare that I have referenced use of GenAI outputs within my assessment in line with the University referencing guidelines.*

***Please note that submitting your work without an accompanying declaration, or one with no ticked boxes, will be considered a declaration that you have not used GenAI tools in preparing your work.***